

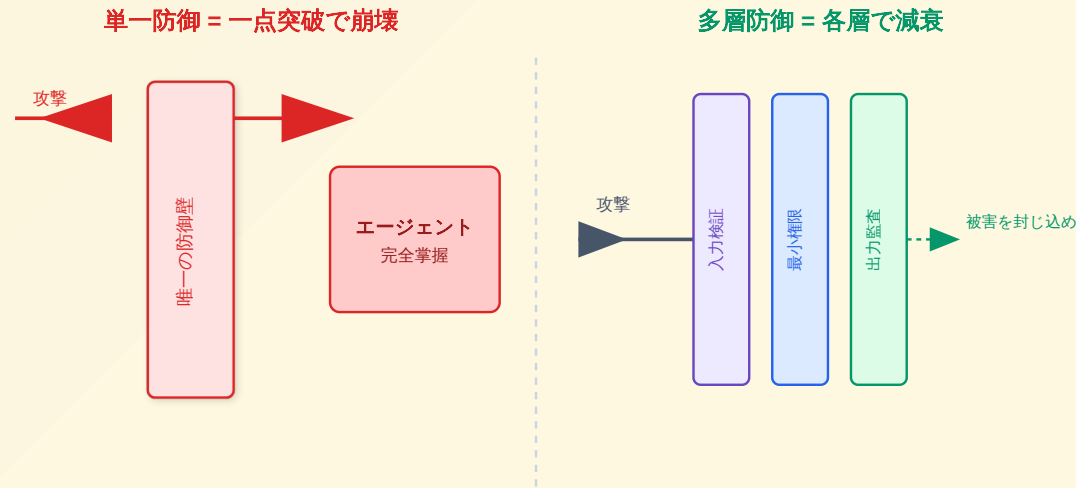
AIエージェントのセキュリティ／ガバナンス 2026

“多層防御で自律エージェントを守る – テックリーダーのための脅威と対策”

- 対象: テックリーダー / エンジニアリングマネージャー
- 自律エージェント時代の攻撃面と、実装できる防御を体系化する

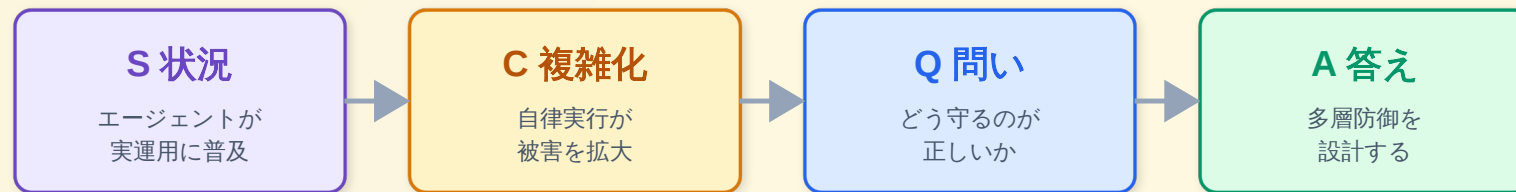
自律エージェントは攻撃を"スケール"させる。防御は多層で設計する

“ 単一の防御線は必ず破られる。入力・ツール・出力の各層で止める前提に立つ ”



本日の道筋: 状況から結論までを一直線でたどる

“ SCQA – なぜ別物か → 脅威カタログ → 多層防御 → ガバナンスの順に進む ”



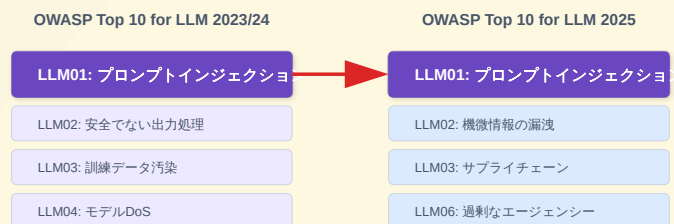
脅威カタログ → 攻撃面 → 多層防御 → ガバナンスと運用

なぜエージェントのセキュリティは別物か

- 従来のアプリと違い、自然言語の入力そのものが"実行される命令"になる
- エージェントは自律的にツールを呼び、外部データを読み、多段で動く

プロンプトインジェクションは OWASP LLMで全版1位を守り続けている

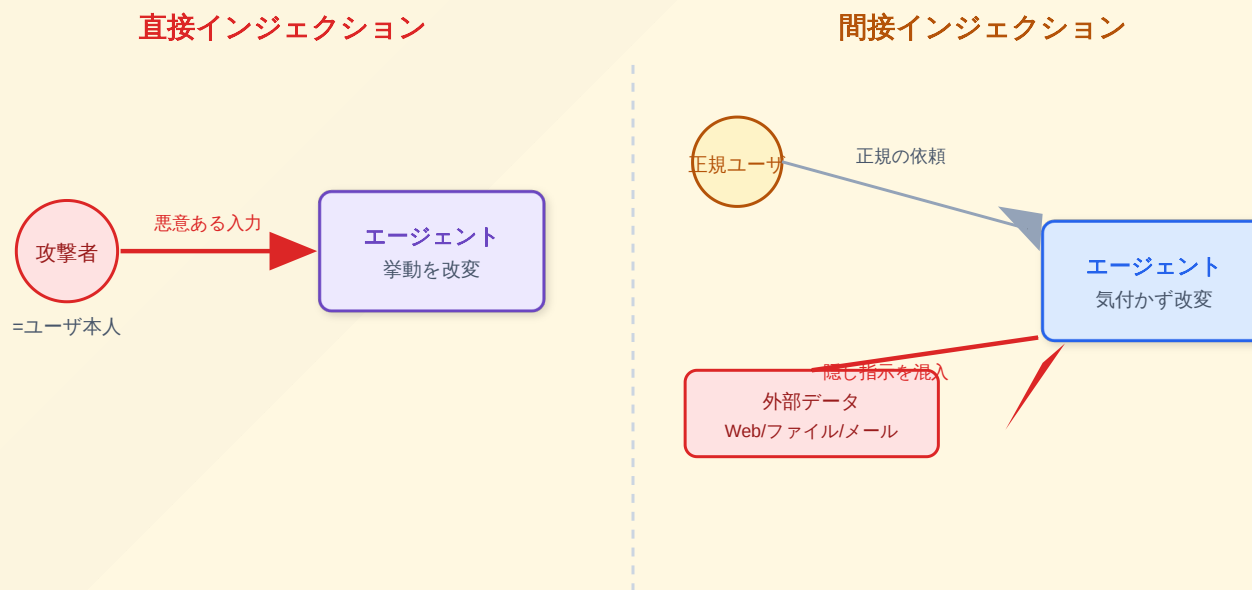
“ 2023/24版でも2025版でもLLM01は不動の首位 – 最優先で対処すべき脅威 ”



版が変わってもLLM01は1位のまま = 構造的な弱点

直接インジェクションと間接インジェクションを区別して防ぐ

“ ユーザ入力経由か、外部データ経由か — 経路が違えば防御の置き所も違う ”



Agent Goal Hijackは「注入」×「過剰自律」で被害が跳ね上がる

“ *ASI01 = LLM01(プロンプトインジェクション)とLLM06(過剰なエージェント)の合成* ”

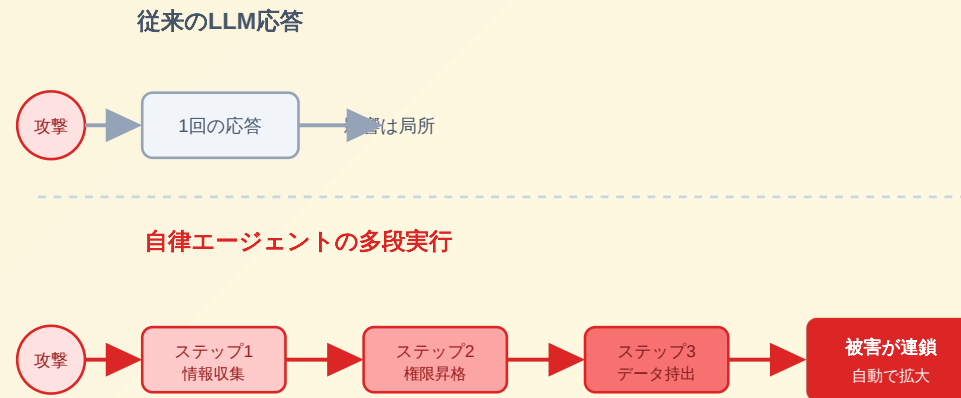


注入された指示が、広い権限で自律実行される = 目標の乗っ取り

OWASP Agentic Security Initiative の脅威分類

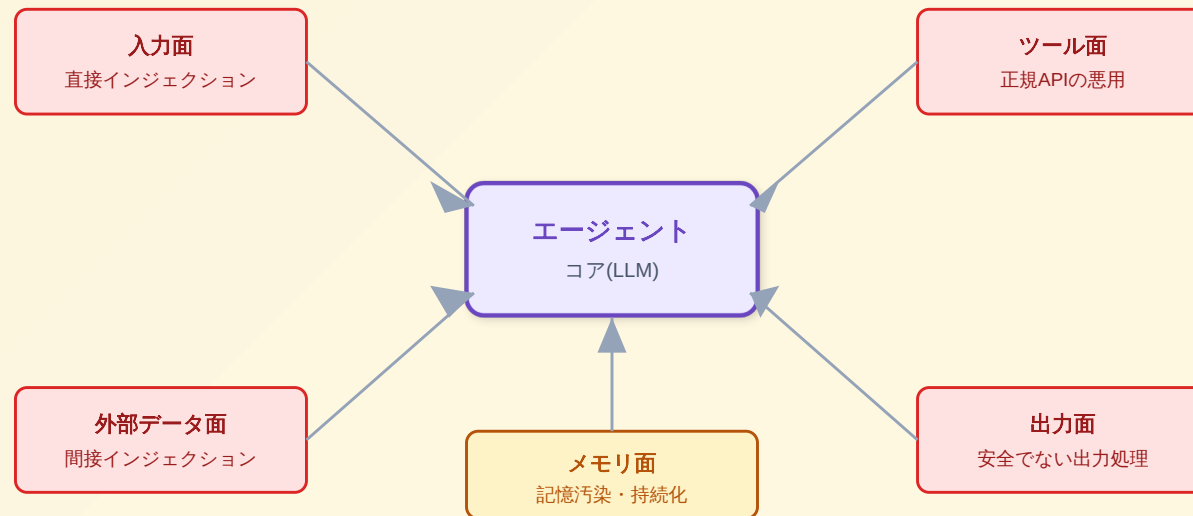
自律的なマルチステップ実行が一回の攻撃を連鎖被害に変える

“従来は「1応答=1被害」。エージェントは多段自動実行で影響が雪だるま式に拡大”



エージェントの攻撃面は入力・ツール・メモリ・外部・出力に広がる

“ 守るべき面は5つ – どこか一つでも無防備なら全体が破られる ”



主要脅威カテゴリー

- OWASP LLM Top10 と Agentic Security の脅威を、実例とともに棚卸しする
- 間接インジェクション・過剰権限・データ漏洩・ツール悪用を順に見る

OWASP LLM Top10とAgents Top10を 1枚で俯瞰する

“汎用LLMの脅威に、自律性由来のエージェント固有脅威が積み上がる構造”



左=モデル層の脅威（濃色=最優先）／右=自律性が生む固有脅威

※項目名は各フレームワークの代表例。番号体系は版により異なる

間接インジェクションはWebやファイル経由で指示を上書きする

“ エージェントが読み込む外部コンテンツに隠された命令が、正規の指示を乗っ取る ”



ユーザは「Web要約」を頼んだだけ。攻撃は本文中に隠れている

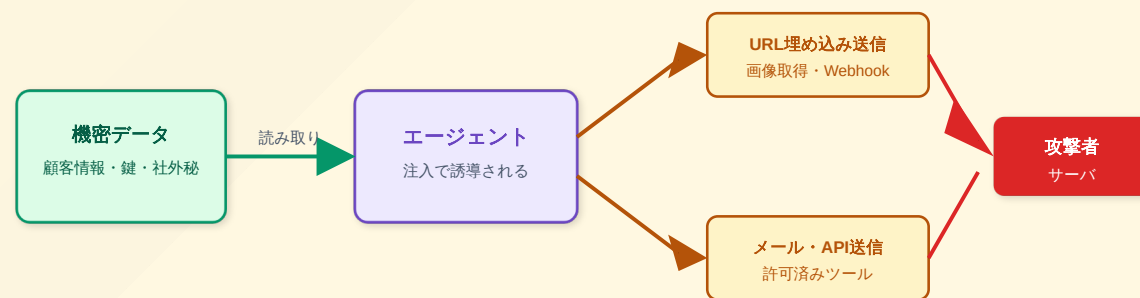
過剰な権限がインシデントを「最悪」に増幅する

“ 同じ注入でも、与えた権限の広さがそのまま被害の上限を決める ”



エージェントは機密を「正規の経路」 で外部へ持ち出しまう

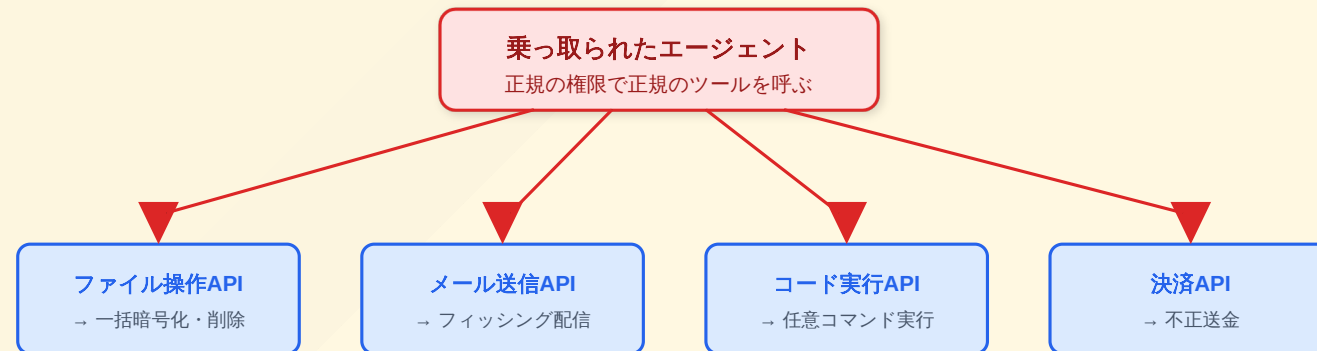
“ 検索・送信・要約といった許可済みツールが、そのまま漏洩チャネルになる ”



漏洩は「不正アクセス」ではなく「許可された動作」として起きる

正規ツールそのものが攻撃の武器に変わる

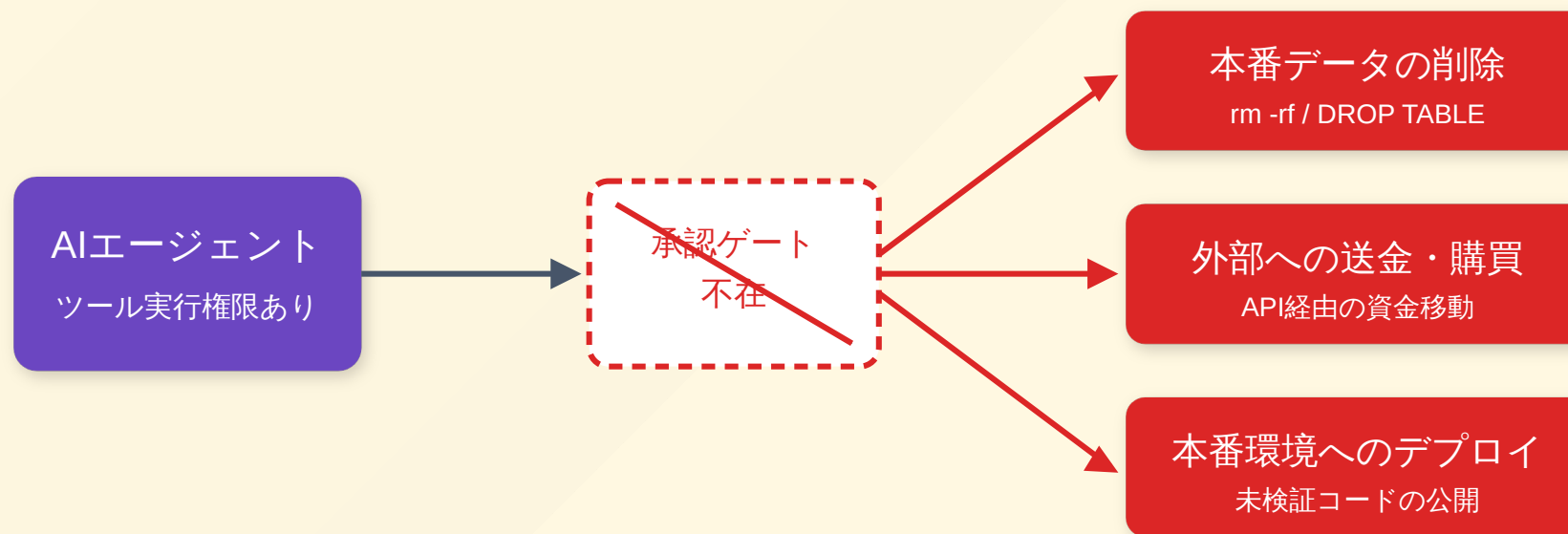
“脆弱性ゼロのAPIでも、悪意ある目的で連続実行されれば攻撃手段になる”



「壊れたAPI」ではなく「意図しない目的で使われたAPI」が脅威
対策=ツール単位の承認・レート制限・破壊的操作の人間承認

承認なしの不可逆操作がエージェント最大の実害を生む

“削除・送金・デプロイは取り消せない。一度実行されれば被害は確定する”

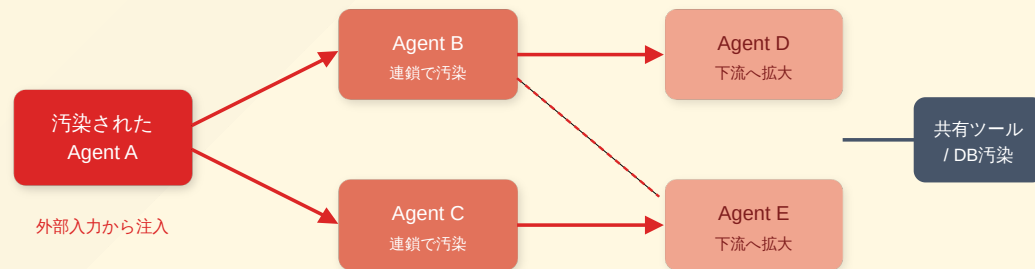


汚染された指示 → 権限をそのまま行使 → 取り消し不能な結果

- OWASP LLM06 「過剰なエージェンシー」－ 権限・自律性・機能の付与しすぎが根本原因

1体の汚染がマルチエージェント全体へ連鎖伝播する

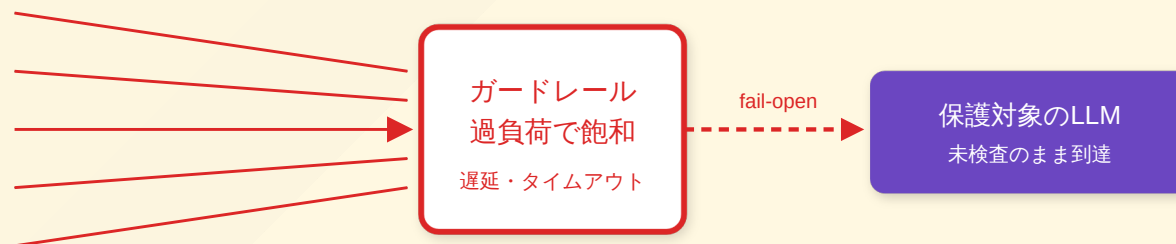
“ エージェント間メッセージが新たな注入経路になり、被害が指数的に拡大する ”



ガードレール自体をDoSで飽和させれば防御は素通しになる

“ 検査層に過負荷をかけタイムアウトさせ、フェイルオープンで攻撃を通す ”

攻撃者：大量リクエスト



検査コストは処理コストの数倍。防御機構こそ可用性の単一障害点になりうる

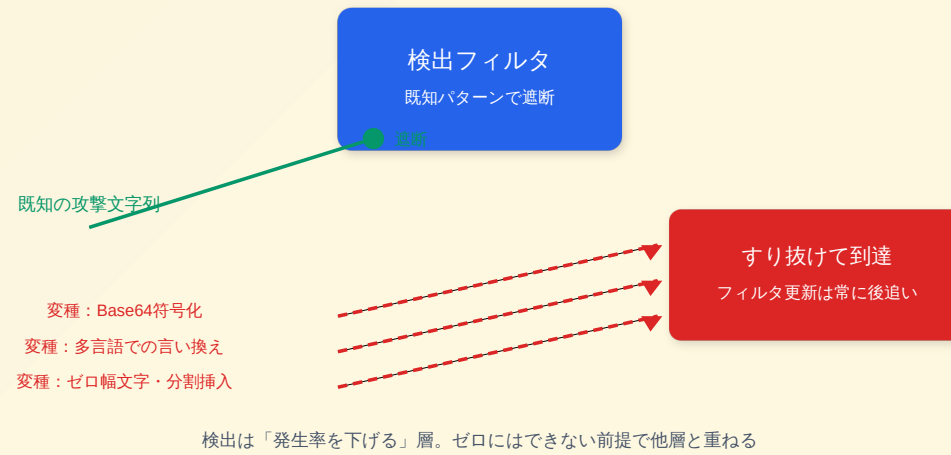
対策：fail-closed（疑わしければ拒否）＋レート制限＋検査のタイムアウト設計

なぜ防ぎきれないのか

- 検知・単層防御の構造的な限界を直視する
- 「完璧な検出」を前提にした設計は必ず破られる

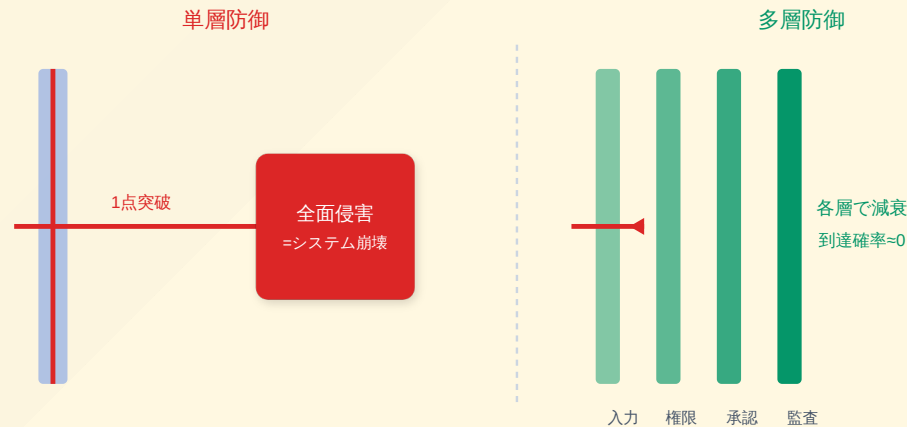
完全なインジェクション検出は原理的に不可能である

“ 自然言語の意味は無限に言い換えられ、フィルタは常に変種にすり抜けられる ”



単層防御は1点突破されれば全体が崩壊する

“多層防御は各層が独立に確率を下げ、全層同時突破の確率を実質ゼロに近づける”



課題の要点

“脅威は「防ぎきれない」前提で設計する – 検出依存から多層防御へ発想を切り替える”

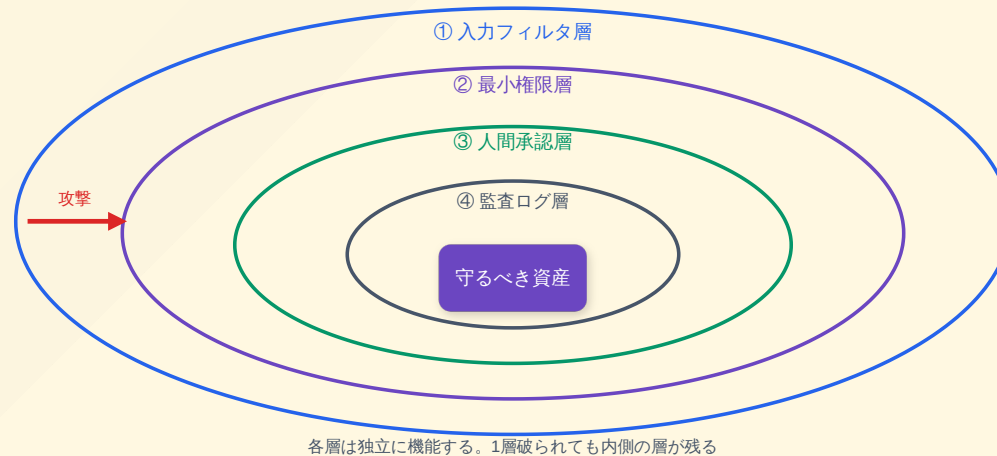
- 不可逆操作の暴走：削除・送金・デプロイは実行された時点で被害が確定する
- 脅威の伝播：マルチエージェントでは1体の汚染が全体へ連鎖する
- 防御機構の脆弱性：ガードレール自体がDoSと検出限界で無力化されうる
- 帰結：単層・検出依存は必ず破られる。層を重ねて被害確率を下げる設計へ

多層防御の設計指針

- 検出をゼロにできない前提で、独立した5つの層を重ねる
- 入力フィルタ・最小権限・入出力検査・人間承認・敵対テスト

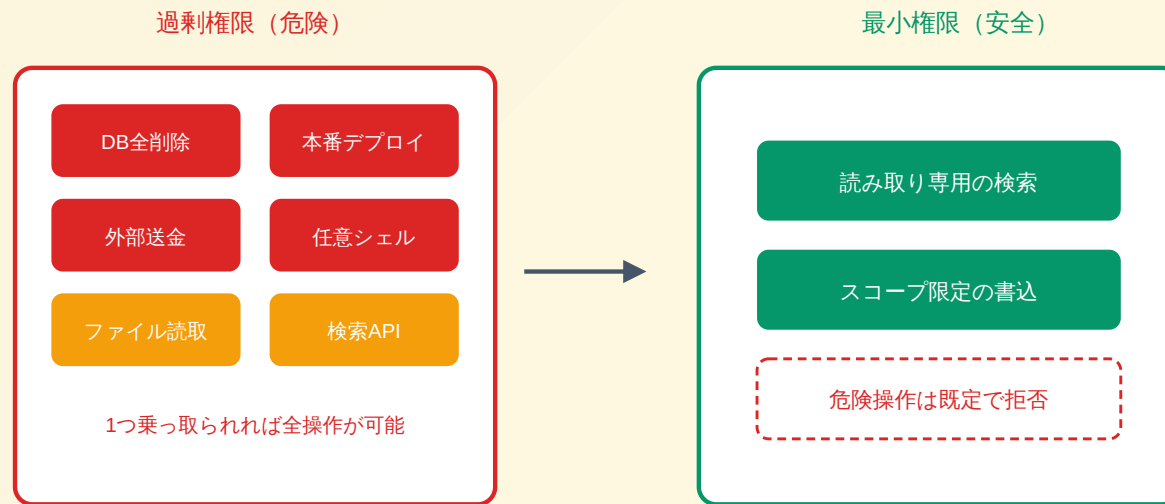
指針①：独立した層を重ね全層同時突破を困難にする

“ *defense-in-depth* – 入力・権限・承認・監査の4層で被害確率を掛け算で下げる ”



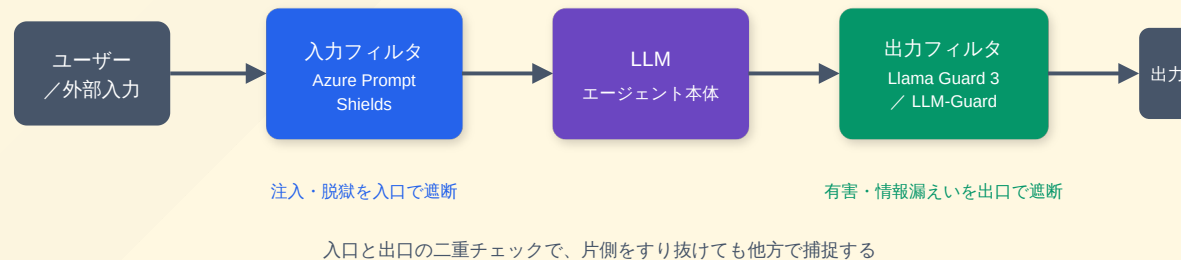
指針②：ツール権限を必要最小に絞れば暴走の影響を封じ込める

“ *least-privilege* – 使う権限だけ許可し、危険操作は既定で拒否する ”



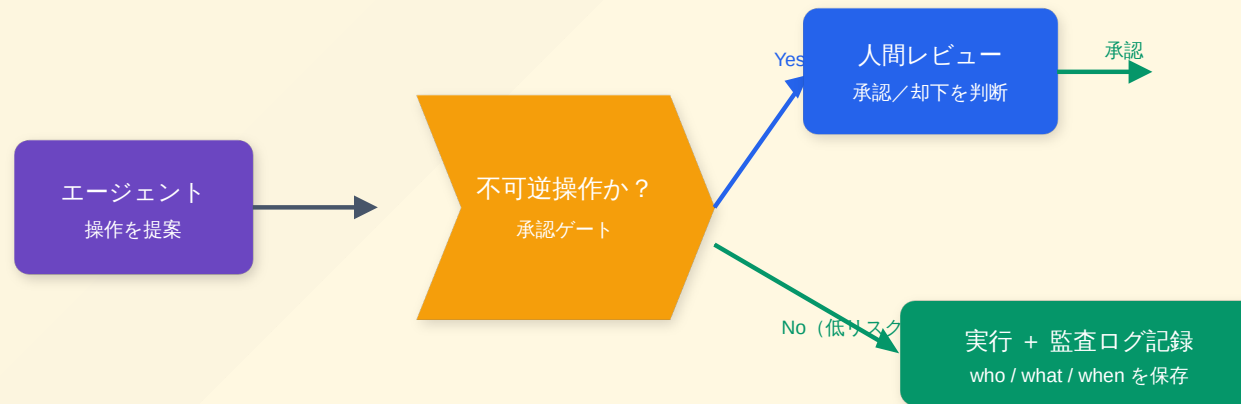
指針③：入出力を専用フィルタで挟み込む

“入力側で*Prompt Shields*、出力側で*Llama Guard 3 / LLM-Guard*が二重に検査する”



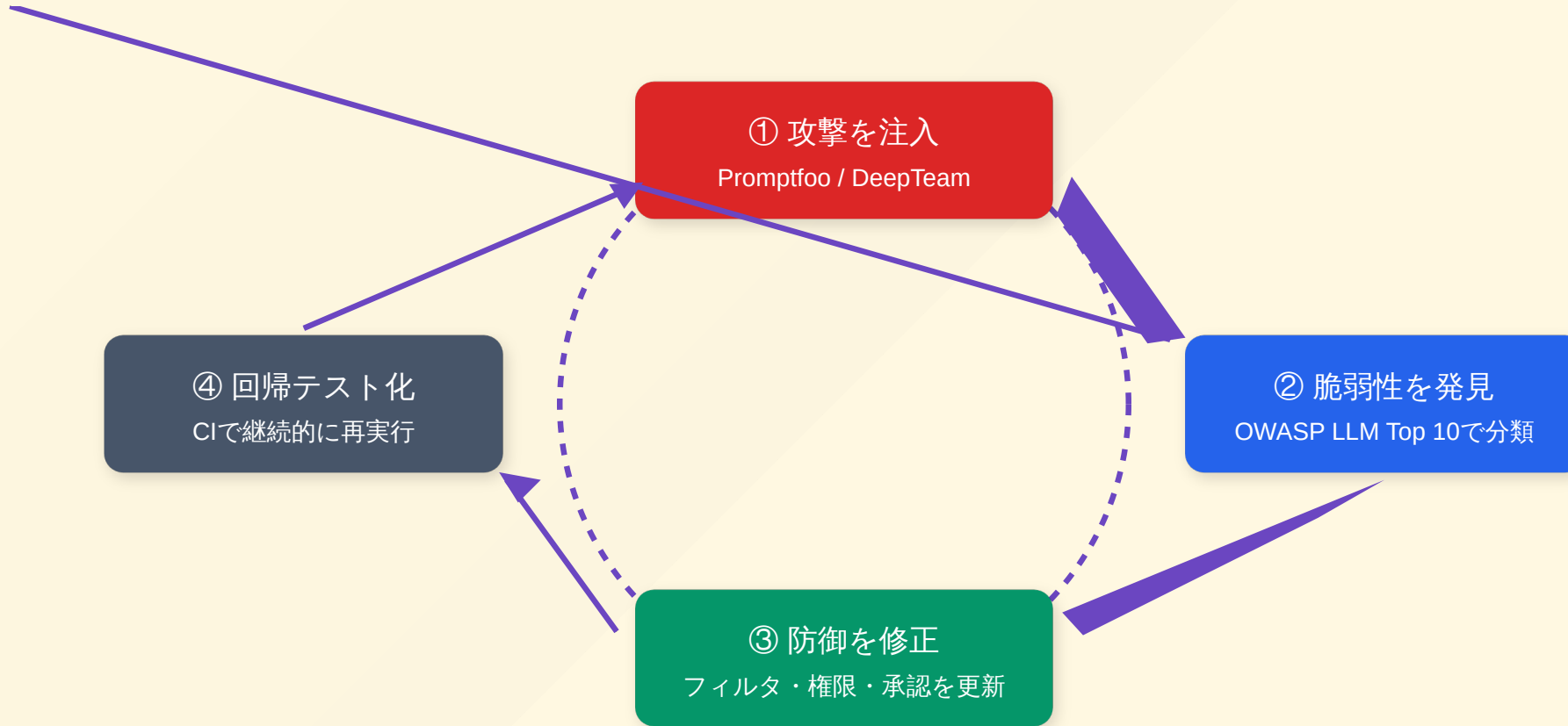
指針④：高リスク操作の前に人間承認ゲートを必ず挟む

“ HITL – 不可逆操作は自動実行させず、人間の明示承認を必須にする ”



指針⑤：敵対テストを定期実行し防御を継続的に鍛える

“ Promptfoo / DeepTeam でレッドチームを自動化し、攻撃→発見→修正を回し続ける ”



- テストはCIに組み込み回帰化する – 一度直した脆弱性を再発させない

導入チェックリスト：5つの防御を点検する

“ 全部を一度にやらない。まず「承認ゲート・最小権限・監査ログ」の3点から始める ”

- 多層防御：入力・権限・承認・監査の各層を独立に配置したか
- 最小権限：ツール権限を必要最小に絞り、危険操作を既定拒否にしたか
- 入出力フィルタ：Prompt Shields／Llama Guard 3等を入口・出口に設置したか
- 人間承認：削除・送金・デプロイ等の不可逆操作にHITLゲートを挟んだか
- レッドチーム：Promptfoo／DeepTeamの敵対テストをCIで定期実行しているか

今すぐ着手する3つのアクション

“全部を一度にやらない。この3点から始める”

- ①不可逆操作（削除・送金・デプロイ）に人間承認ゲートを挟む
- ②ツール権限を棚卸しし、必要最小まで縮小する
- ③全操作を監査ログ化し、追跡可能にする

まとめ：多層防御でエージェントを守る

“ 検出は破られる前提で、独立した層を重ねて被害確率を下げる – これが唯一の現実解

”

- 検出は破られる前提で、独立した層を重ねて被害確率を下げる – これが唯一の現実解
- 今日から：不可逆操作に人間承認、ツールは最小権限、全操作を監査ログへ

参考文献・出典

- 以下の公開情報を参照
- OWASP LLM Top 10 – Promptfoo: <https://www.promptfoo.dev/docs/red-team/owasp-llm-top-10/>
- Prompt Injection in 2026: OWASP's #1 LLM Threat: <https://www.kunalganglani.com/blog/prompt-injection-2026-owasp-llm-vulnerability>
- OWASP Top 10 for Agents 2026 – DeepTeam: <https://www.trydeepteam.com/docs/frameworks-owasp-top-10-for-agentic-applications>
- OWASP Top 10 for LLM Applications (2025) – Aembit: <https://aembit.io/blog/owasp-top-10-llm-risks-explained/>